

Parte 3: Comandos Linux + Git

Valor 2.5 ptos cada pregunta. **Total: 25 puntos**

INSTRUCCIONES DE ENTREGA

En una carpeta comprimida en **.zip** con nombre parte3.zip debes entregar:

- Tu directorio creado.
- **URL pública** de tu repositorio en GitHub
- Lista de comandos utilizados en la parte 3.1 (bash-linux) - Usar `history > historial_linux.txt`
- Lista de comandos utilizados en la parte 3.2 (git) - `history > historial_gitbash.txt`

Vas a crear y gestionar la estructura de directorios de un proyecto completo de Ingeniería de Datos y Big Data. Lee cuidadosamente cada pregunta y ejecuta los comandos necesarios.

Parte 3.1 - Comandos Linux

Pregunta 1: Creación de la estructura base del proyecto

Crea la siguiente estructura de directorios para un proyecto de Big Data llamado `bigdata_pipeline_project`:

```
bigdata_pipeline_project/
├── data/
│   ├── raw/
│   │   ├── streaming/
│   │   └── batch/
│   ├── processed/
│   │   ├── bronze/
│   │   ├── silver/
│   │   └── gold/
│   └── archive/
├── etl/
│   ├── extract/
│   ├── transform/
│   └── load/
├── pipelines/
│   ├── airflow/
│   └── spark/
├── notebooks/
├── src/
│   ├── ingestion/
│   ├── quality/
│   └── utils/
├── config/
├── schemas/
├── logs/
├── tests/
└── outputs/
```

Tareas:

- Crea toda la estructura con el mínimo número de comandos posible

- Verifica que la estructura se creó correctamente mostrando el árbol de directorios (usa `tree` o `ls -R`)

Pregunta 2: Creación de archivos de configuración y documentación

Dentro del proyecto, crea los siguientes archivos:

1. En la raíz del proyecto:
 - `README.md` con el texto: "# Big Data Pipeline Project - Data Engineering"
 - `requirements.txt` con las librerías: `pyspark`, `apache-airflow`, `kafka-python`, `pandas`, `pyarrow`
 - `.gitignore` con las entradas: `.pyc` , `__pycache__` / , `logs/` , `.log`
2. En `config/` :
 - `database.yaml` (archivo vacío)
 - `spark.conf` (archivo vacío)

Tareas:

- Crea todos los archivos usando comandos de terminal
- Muestra el contenido del archivo `README.md` y `requirements.txt`

Pregunta 3: Simulación de datos y archivos de log (2 puntos)

Tareas:

1. En `data/raw/batch/` , crea 3 archivos de datos:
 - `sales_2024_Q1.csv`
 - `sales_2024_Q2.csv`
 - `customers.json`
2. En `data/raw/streaming/` , crea:
 - `events_01.json`
 - `events_02.json`
3. En `logs/` , crea archivos de log:
 - `pipeline_2024-01-15.log`
 - `pipeline_2024-02-20.log`
4. Añade contenido a `pipeline_2024-01-15.log` con el texto:

```
[INFO] ETL Pipeline started
[INFO] Processing 500,000 records
[SUCCESS] Pipeline completed
```

5. Lista todos los archivos `.log` del proyecto de forma recursiva

Pregunta 4: Búsqueda y filtrado de archivos

Tareas:

1. Encuentra todos los archivos `.csv` en el proyecto y muestra sus rutas completas
2. Cuenta cuántos archivos `.json` existen en total en el proyecto
3. Busca recursivamente todos los directorios que contengan la palabra "data" en su nombre
4. Lista únicamente los archivos (no directorios) que están dentro de `data/raw/` y sus subdirectorios

Pregunta 5: Operaciones de copia y movimiento

Simula el flujo de trabajo de procesamiento de datos en una arquitectura medallion (bronze → silver → gold):

Tareas:

1. Copia `sales_2024_Q1.csv` desde `data/raw/batch/` hacia `data/processed/bronze/` manteniendo el mismo nombre
 2. Copia el mismo archivo desde `data/processed/bronze/` hacia `data/processed/silver/` renombrándolo como `sales_2024_Q1_validated.csv`
 3. Mueve todos los archivos `.json` desde `data/raw/streaming/` hacia `data/archive/`
 4. Verifica las operaciones listando el contenido de:
 - `data/processed/bronze/`
 - `data/processed/silver/`
 - `data/archive/`
-

Parte 3.2 - Control de Versiones

Pregunta 6: Inicialización y primer commit

Tareas:

1. Inicializa un repositorio Git en el directorio `bigdata_pipeline_project`
 2. Añade todos los archivos al staging area
 3. Realiza el primer commit con el mensaje: "Initial commit: Big Data project structure"
 4. Verifica el historial de commits
-

Pregunta 7: Conexión con GitHub y push

Tareas:

1. Crea un repositorio vacío en GitHub llamado `bigdata-pipeline-project`
 2. Conecta tu repositorio local con el remoto.
 3. Sube la rama principal al repositorio remoto.
 4. Verifica en GitHub que los archivos se subieron correctamente
-

Pregunta 8: Crear rama y hacer cambios

Tareas:

1. Crea y cámbiate a una nueva rama llamada `feature/documentation`
 2. Añade un archivo `CONTRIBUTING.md` en la raíz con el texto: "# Contributing Guidelines"
 3. Modifica `README.md` añadiendo al final: "## Version: 1.0.0"
 4. Haz commit de los cambios con el mensaje: "Add documentation"
 5. Sube la rama al repositorio remoto
-

Pregunta 9: Fusión mediante Pull Request

Tareas:

1. Ve a GitHub y crea un Pull Request desde `feature/documentation` hacia `main`
 2. Añade la descripción: "Adding project documentation"
 3. Fusiona (merge) el Pull Request
 4. Opcionalmente elimina la rama remota después del merge
-

Pregunta 10: Actualizar repositorio local

Tareas:

1. Regresa a la rama `main` en tu repositorio local
2. Actualiza tu rama local con los cambios del remoto

3. Verifica que los cambios del merge están presentes con `git log`

4. Muestra el contenido actualizado del `README.md`

INSTRUCCIONES DE ENTREGA

En una carpeta comprimida en **.zip** con nombre parte3.zip debes entregar:

- Tu directorio creado.
- **URL pública** de tu repositorio en GitHub
- Lista de comandos utilizados en la parte 3.1 (bash-linux) - Usar `history > historial_linux.txt`
- Lista de comandos utilizados en la parte 3.2 (git) - `history > historial_gitbash.txt`